

星露谷战斗模块

俯视角像素风动作角色扮演游戏

— 以《星露谷物语》矿洞战斗为灵感 —

项目设计与实现文档

课程名称： 软件项目开发与实践

项目名称： 星露谷战斗模块

姓 名： 谢宇鹏

学 号： 5120243390

开发模式： B 模式（AI 智能协作开发）

技术栈： Python + Pygame

文档版本： V1.0

本文档阐述系统架构设计、面向对象类结构、核心算法分析和测试方案。

目录

1	系统架构设计	3
1.1	架构风格选择	3
1.2	系统组件图	3
1.3	分层职责说明	4
2	面向对象设计	4
2.1	设计原则应用	4
2.2	类图设计	4
2.3	类职责详述	4
2.3.1	游戏精灵基类 (Sprite)	4
2.3.2	实体基类 (Entity)	6
2.3.3	玩家类 (Player)	6
2.3.4	敌人类 (Enemy)	6
2.3.5	关卡管理类 (Level)	6
3	核心算法分析	7
3.1	智能寻路算法	7
3.1.1	问题建模	7
3.1.2	候选算法分析	7
3.1.3	算法选型对比	8
3.1.4	A* 算法伪代码	8
3.1.5	性能优化策略	8
3.2	分轴 AABB 碰撞检测	10
3.2.1	问题分析	10
3.2.2	解决方案	10
3.3	Y 轴排序渲染	10
3.3.1	问题分析	10
3.3.2	解决方案	10
3.4	有限状态机 (FSM)	10
3.4.1	敌人 AI 状态机	10
3.4.2	游戏主流程状态机	12
4	关键交互时序	12
4.1	近战攻击时序图	12
4.2	场景切换时序图	12

4.3 怪物 AI 行为时序图	17
5 数据结构设计	17
5.1 配置数据结构	17
5.2 地图数据结构	17
5.3 存档数据结构	17
6 场景交互设计	18
6.1 场景状态转换图	18
6.2 各场景详细设计	18
6.2.1 主菜单场景	18
6.2.2 游戏关卡场景	18
6.2.3 死亡界面	20
7 测试方案	20
7.1 功能测试	20
7.2 边界测试	21
7.3 测试策略	22
8 项目统计	22
参考文献	22

第 1 章 系统架构设计

架构风格选择

经过对多种架构风格的对比分析，本项目采用**分层架构**（Layered Architecture），结合面向对象设计原则。

架构风格	优点	缺点
分层架构	职责清晰，上下层解耦，易于测试和维护	层间通信有一定开销
MVC	模型视图分离	对实时游戏循环不太自然
ECS	高性能，数据驱动	实现复杂度高，不适合小项目
事件驱动	松耦合，扩展性好	调试困难，流程不直观

选择理由： 分层架构最适合本项目的规模和复杂度——层级清晰、职责明确、团队协作方便。

系统组件图

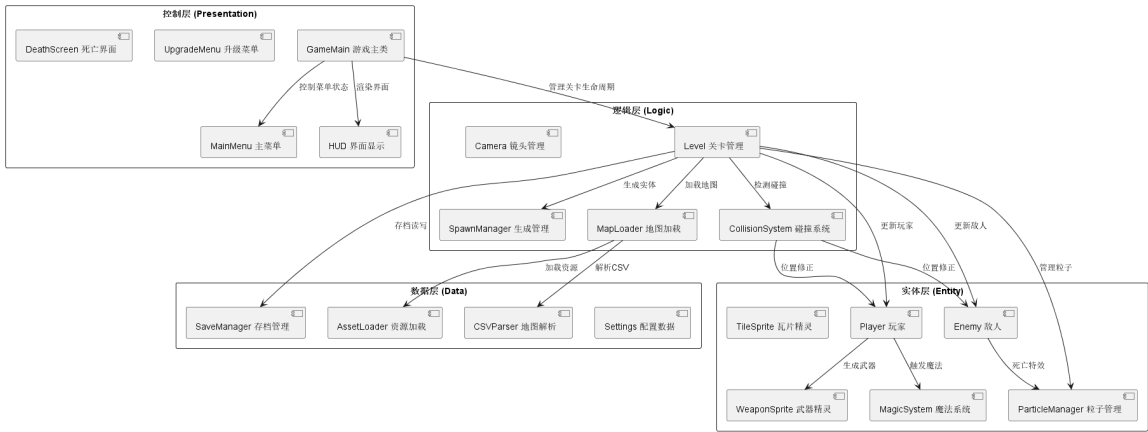


图 1: 系统组件图（PlantUML 生成）

- 系统分为四层：
- **控制层：** GameMain 游戏主类、MainMenu 主菜单、HUD 界面显示、UpgradeMenu 升级菜单、DeathScreen 死亡界面
 - **逻辑层：** Level 关卡管理、MapLoader 地图加载、Camera 镜头管理、CollisionSystem 碰撞系统、SpawnManager 生成管理

- **实体层**: Player 玩家、Enemy 敌人、WeaponSprite 武器精灵、MagicSystem 魔法系统、ParticleManager 粒子管理、TileSprite 瓦片精灵
- **数据层**: Settings 配置数据、SaveManager 存档管理、CSVParser 地图解析、AssetLoader 资源加载

分层职责说明

层级	职责	包含模块
控制层	游戏主循环、状态机、输入事件、菜单渲染	GameMain, MainMenu, HUD, UpgradeMenu, DeathScreen
逻辑层	关卡管理、地图加载、碰撞检测、场景切换、实体生成	Level, MapLoader, Camera, CollisionSystem, SpawnManager
实体层	玩家、敌人、武器、魔法、粒子、瓦片精灵	Player, Enemy, WeaponSprite, MagicSystem, ParticleManager, TileSprite
数据层	配置数据、存档读写、地图解析、资源加载	Settings, SaveManager, CSVParser, AssetLoader

设计原则:

1. 上层依赖下层，下层不依赖上层（依赖倒置）
2. 同层之间通过回调函数通信，避免直接引用
3. 每个模块职责单一，符合 SRP 原则

第2章 面向对象设计

设计原则应用

类图设计

类职责详述

游戏精灵基类 (Sprite)

Pygame 的 Sprite 基类的封装，提供统一的图像加载和矩形更新接口。所有可视对象继承此类。

原则	在本项目中的体现	设计目的
封装	每个类管理自己的状态，外部通过方法交互	隐藏实现细节，降低耦合
继承	玩家和敌人继承实体基类，复用移动和碰撞代码	代码复用，减少重复
多态	同一移动方法在不同子类中表现不同（键盘驱动 vs AI 驱动）	统一接口，灵活扩展
组合	关卡类组合管理玩家、敌人、武器等对象	灵活装配，职责分离
单一职责	每个类只负责一个功能域	降低复杂度，便于维护

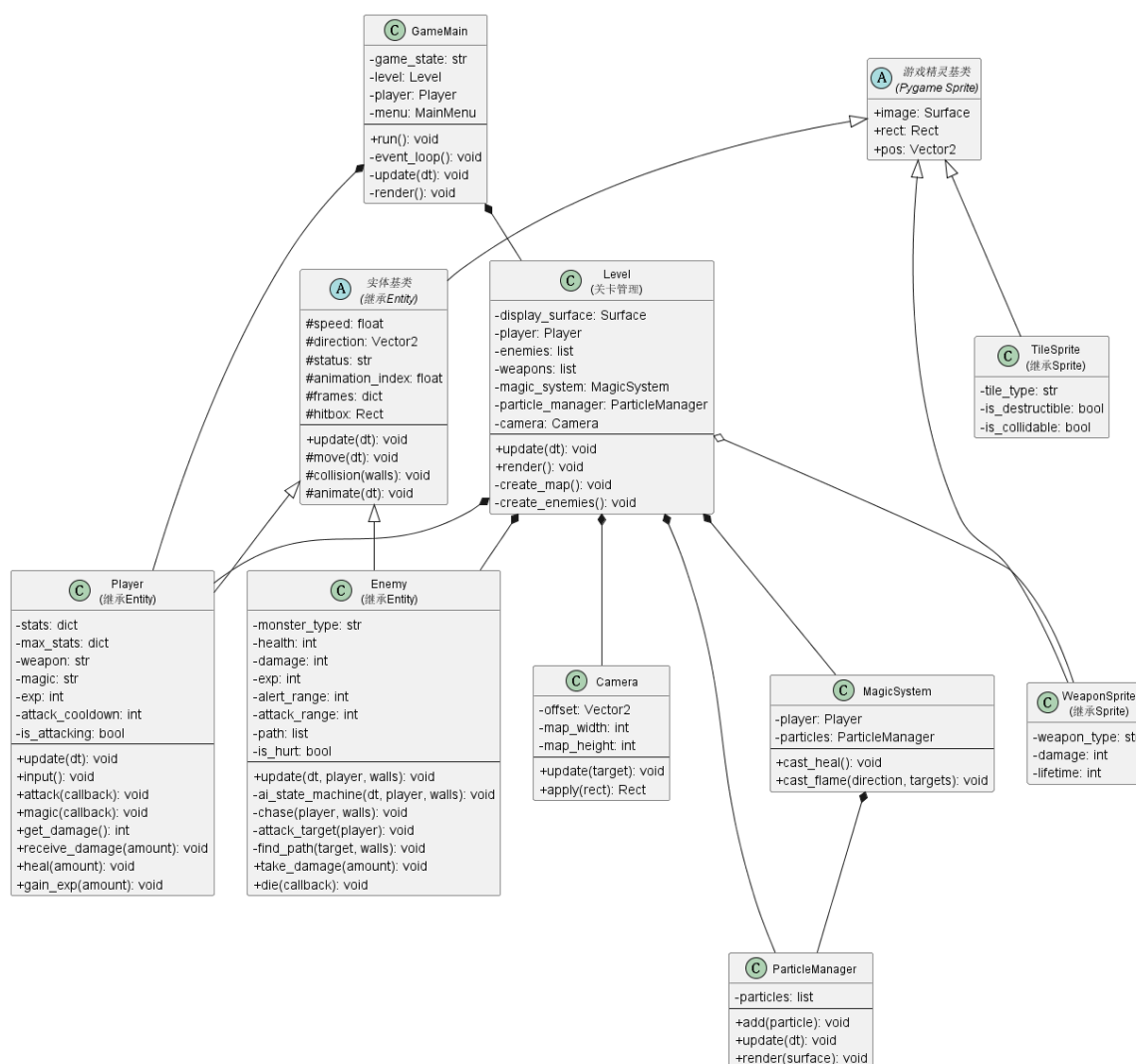


图 2: 类图——继承与组合关系 (PlantUML 生成)

实体基类 (Entity)

封装公共行为:

- **统一移动**: 速度 $\times$ 方向向量 $\times$ 帧时间
- **分轴碰撞**: 先水平后垂直, 避免对角线穿墙
- **Hitbox**: 缩小碰撞框, 比精灵图像小一圈, 提升碰撞精度
- **动画帧管理**: 浮点索引累加, 支持不同帧数的动画混合

玩家类 (Player)

输入驱动型角色:

- 键盘输入 $\rightarrow$ 设置方向 $\rightarrow$ 切换动画状态
- 攻击/魔法触发通过回调通知关卡层
- 冷却管理通过时间戳差值判断
- 能量每帧自动恢复

敌人类 (Enemy)

AI 驱动型角色:

- 三态状态机 (待机/追踪/攻击), 根据玩家距离切换
- 追踪时调用寻路算法, 路径缓存 + 定时重算
- 死亡时通知关卡层播放特效、增加经验值

关卡管理类 (Level)

核心协调者:

- 地图解析 $\rightarrow$ 实体创建 $\rightarrow$ 碰撞调度 $\rightarrow$ 传送管理 $\rightarrow$ 存档接口
- 管理所有实体的更新和渲染顺序
- 处理玩家攻击和魔法的回调

第3章 核心算法分析

智能寻路算法

问题建模

敌人需要在网格地图上找到一条从自身位置到玩家位置的路径，且该路径必须绕过障碍物（树木、石墙等不可通行格子）。

设地图为 $M \times N$ 的网格，每个格子状态为「可通行」或「不可通行」。起点为敌人位置 S ，终点为玩家位置 G 。目标：找到从 S 到 G 的最短路径（步数最少）。

候选算法分析

候选一：广度优先搜索（BFS） 从起点开始逐层扩展，先访问所有距离为 1 的节点，再访问距离为 2 的节点，直到找到终点。

- 优点：一定能找到最短路径，实现简单，无需设计启发函数
- 缺点：无方向性，盲目扩展所有方向（上/下/左/右/对角线），搜索范围大
- 时间复杂度： $O(V + E)$ ， V 为节点数， E 为边数
- 空间复杂度： $O(V)$
- 适用性：适合无权图的最短路径，但效率偏低

候选二：Dijkstra 算法 从起点开始，每次选择当前累计代价最小的节点扩展，逐步到达终点。

- 优点：保证最短路径，支持加权图（不同地形移动代价不同）
- 缺点：无启发式引导，搜索范围仍较大，向所有方向均匀扩展
- 时间复杂度： $O((V + E) \log V)$ ，使用优先队列
- 适用性：本项目网格权重相同，Dijkstra 的加权能力用不上

候选三：A* 算法（最终选择） 综合 Dijkstra 和贪心搜索，使用评估函数引导搜索方向：

$$f(n) = g(n) + h(n)$$

其中 $g(n)$ 为起点到当前节点的实际代价， $h(n)$ 为当前节点到终点的启发式估计。

- 启发函数：曼哈顿距离 $h(n) = |x_1 - x_2| + |y_1 - y_2|$

- **优点：**保证最短路径（启发函数满足一致性条件时），启发式剪枝大幅减少搜索范围
- **缺点：**需要设计合适的启发函数，最坏情况退化为 Dijkstra
- **时间复杂度：**最坏 $O(E)$ ，实际远小于此
- **空间复杂度：** $O(V)$
- **适用性：**本项目的最佳选择——网格地图、静态障碍物、需最短路径

候选四：贪心最佳优先搜索 每次只考虑启发函数 $h(n)$ ，选择离终点最近的节点扩展。

- **优点：**搜索速度极快，直奔目标方向
- **缺点：**不保证最短路径，可能走入死胡同后需要回溯
- **时间复杂度：** $O(E)$
- **适用性：**速度最快但路径质量无保证，不适合本项目

算法选型对比

算法	最短路径	搜索效率	实现难度	动态地图	本项目适用性
BFS	保证	低	简单	支持	不推荐
Dijkstra	保证	中	中等	支持	可用
A*	保证	高	中等	支持	最终选择
贪心搜索	不保证	极高	简单	支持	不推荐

A* 算法伪代码

性能优化策略

1. **路径缓存：**计算好的路径保存起来，每隔 500ms 或玩家移动到新网格时才重新计算，避免每帧计算
2. **搜索范围限制：**当搜索距离超过一定阈值时停止，防止在大地图上搜索耗时过长
3. **曼哈顿距离：**使用曼哈顿距离而非欧几里得距离，减少浮点运算开销

Algorithm 1 A* 寻路算法

Require: 起点 S , 终点 G , 地图 M **Ensure:** 从 S 到 G 的最短路径

```

1: open_list  $\leftarrow$  优先队列, 按  $f(n)$  排序
2: closed_set  $\leftarrow$  空集合
3:  $g(S) \leftarrow 0, f(S) \leftarrow h(S)$ 
4: 将  $S$  加入 open_list
5: while open_list 非空 do
6:    $current \leftarrow$  open_list 中  $f$  值最小的节点
7:   if  $current = G$  then
8:     return 回溯重建路径
9:   end if
10:  将  $current$  从 open_list 移入 closed_set
11:  for 每个相邻节点  $neighbor$  do
12:    if  $neighbor$  不可通行或  $neighbor \in closed\_set$  then
13:      continue
14:    end if
15:     $tentative\_g \leftarrow g(current) + 1$ 
16:    if  $neighbor \notin open\_list$  或  $tentative\_g < g(neighbor)$  then
17:       $g(neighbor) \leftarrow tentative\_g$ 
18:       $f(neighbor) \leftarrow g(neighbor) + h(neighbor)$ 
19:      记录  $neighbor$  的父节点为  $current$ 
20:      将  $neighbor$  加入 open_list
21:    end if
22:  end for
23: end while
24: return 空路径 (无法到达)

```

分轴 AABB 碰撞检测

问题分析

在 2D 游戏中，角色移动通常是同时更新 x 和 y 坐标。如果在一次碰撞检测中同时处理两个轴，角色可能在对角线方向同时穿过两个相邻障碍物的夹角。

解决方案

将二维碰撞分解为两个一维碰撞，分两步处理：

1. 水平方向：更新 x 坐标后，检测水平方向碰撞，若碰撞则将 x 回退到障碍物边界
2. 垂直方向：更新 y 坐标后，检测垂直方向碰撞，若碰撞则将 y 回退到障碍物边界

为什么分轴：若同时处理两个轴，角色在 $(v_x, v_y) = (1, 1)$ 方向移动时，可能同时穿过水平和垂直方向的两个障碍物夹角。分轴处理从根本上避免了对角线穿墙问题。

Y 轴排序渲染

问题分析

在俯视角游戏中，为了实现正确的遮挡关系（画面下方的角色挡住上方的角色），需要按照 Y 坐标排序后绘制。

解决方案

1. 计算相机偏移量：以玩家位置为中心
2. 绘制地面贴图（最底层）
3. 将所有可见精灵（玩家、敌人、草地、瓦片等）收集到一个列表
4. 按 Y 坐标排序（Y 值大的排后面）
5. 按排序顺序依次绘制，Y 值大的后绘制（覆盖 Y 值小的）

有限状态机（FSM）

敌人 AI 状态机

敌人 AI 包含五个状态：待机、追踪、攻击、受击、死亡。状态转换由玩家距离和自身 HP 决定。

Algorithm 2 分轴碰撞检测

Require: 角色位置 (x, y) , 速度 (v_x, v_y) , 墙壁集合 W , 帧时间 Δt **Ensure:** 修正后的位置 (x', y')

```

1:  $x \leftarrow x + v_x \cdot \Delta t$                                 ▷ 更新水平位置
2:  $hitbox.x \leftarrow x$                                     ▷ 更新碰撞框
3: for 每个墙壁  $wall \in W$  do
4:   if  $hitbox$  与  $wall.rect$  重叠 then
5:     if  $v_x > 0$  then                                    ▷ 向右移动
6:        $hitbox.right \leftarrow wall.rect.left$ 
7:     else if  $v_x < 0$  then                                ▷ 向左移动
8:        $hitbox.left \leftarrow wall.rect.right$ 
9:     end if
10:  end if
11: end for
12:  $x \leftarrow hitbox.x - hitbox\_offset.x$                 ▷ 同步位置
13:  $y \leftarrow y + v_y \cdot \Delta t$                         ▷ 更新垂直位置
14:  $hitbox.y \leftarrow y$                                     ▷ 更新碰撞框
15: for 每个墙壁  $wall \in W$  do
16:   if  $hitbox$  与  $wall.rect$  重叠 then
17:     if  $v_y > 0$  then                                    ▷ 向下移动
18:        $hitbox.bottom \leftarrow wall.rect.top$ 
19:     else if  $v_y < 0$  then                                ▷ 向上移动
20:        $hitbox.top \leftarrow wall.rect.bottom$ 
21:     end if
22:   end if
23: end for
24:  $y \leftarrow hitbox.y - hitbox\_offset.y$                 ▷ 同步位置
25: return  $(x, y)$ 

```

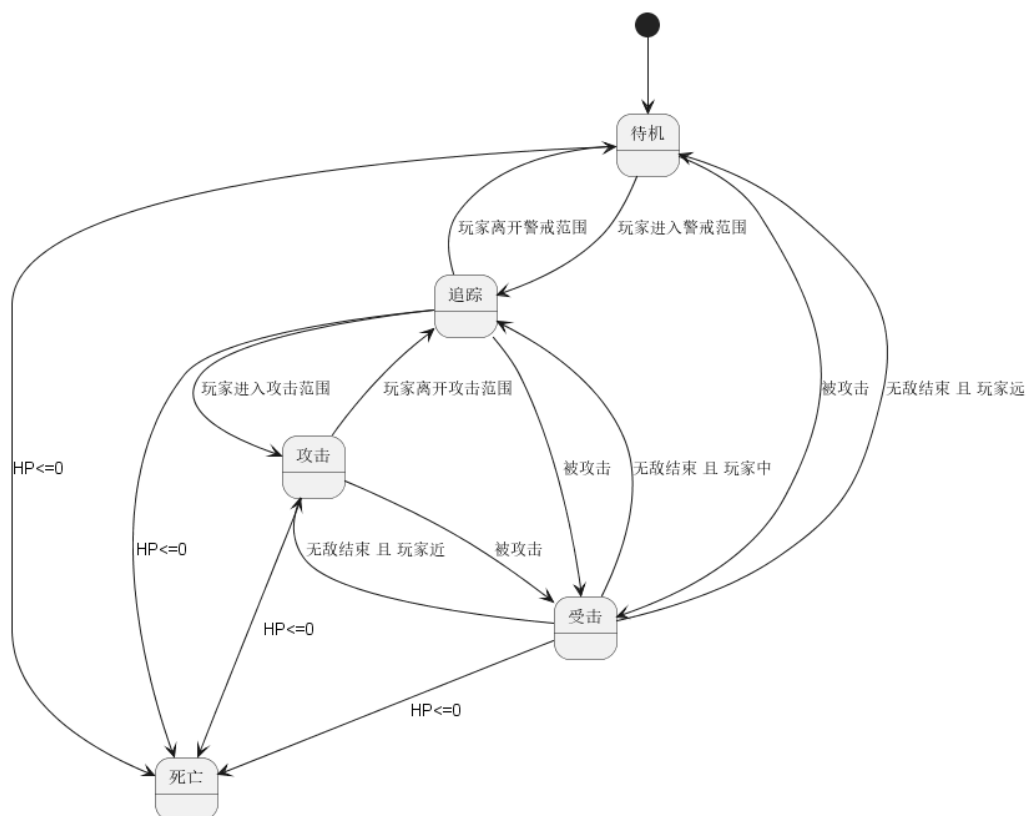


图 3: 敌人 AI 有限状态机 (PlantUML 生成)

游戏主流程状态机

游戏主流程包含四个状态：主菜单、游戏中、死亡界面、升级菜单。

第 4 章 关键交互时序

近战攻击时序图

攻击流程：玩家按空格 → Player 检测冷却 → 回调 Level 创建武器精灵 → 每帧检测碰撞 → 碰撞发生则 Enemy 扣血 → 冷却结束移除武器。

场景切换时序图

切换流程：检测传送点 → 回调 GameMain → 淡出动画 → 销毁旧关卡 → 创建新 Level → 加载新地图 → 淡入动画。

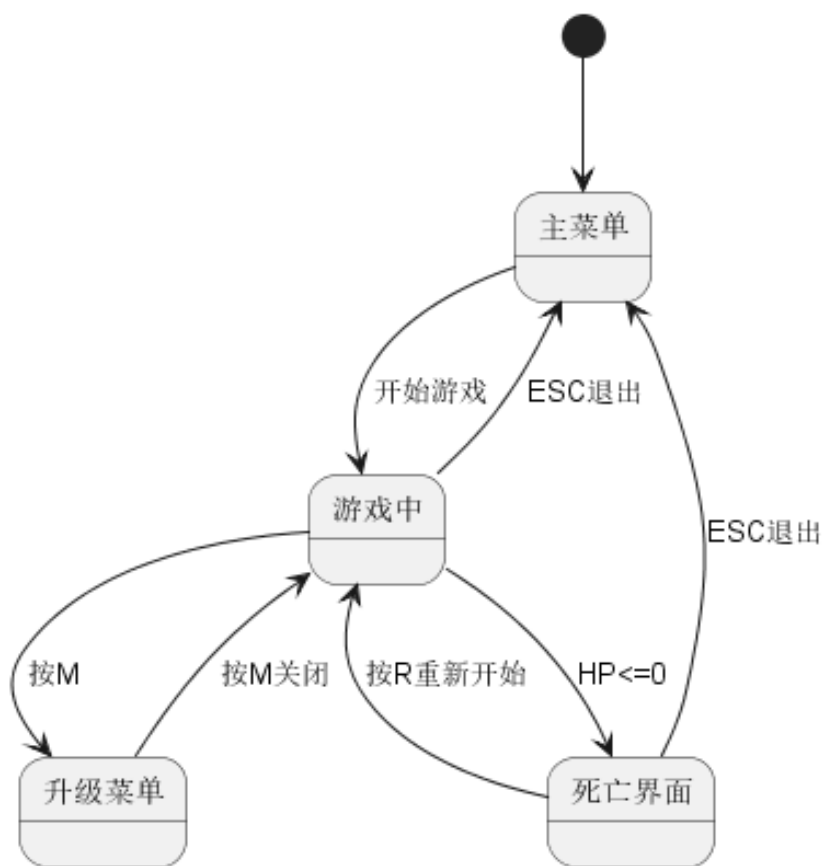


图 4: 游戏主流程状态机 (PlantUML 生成)

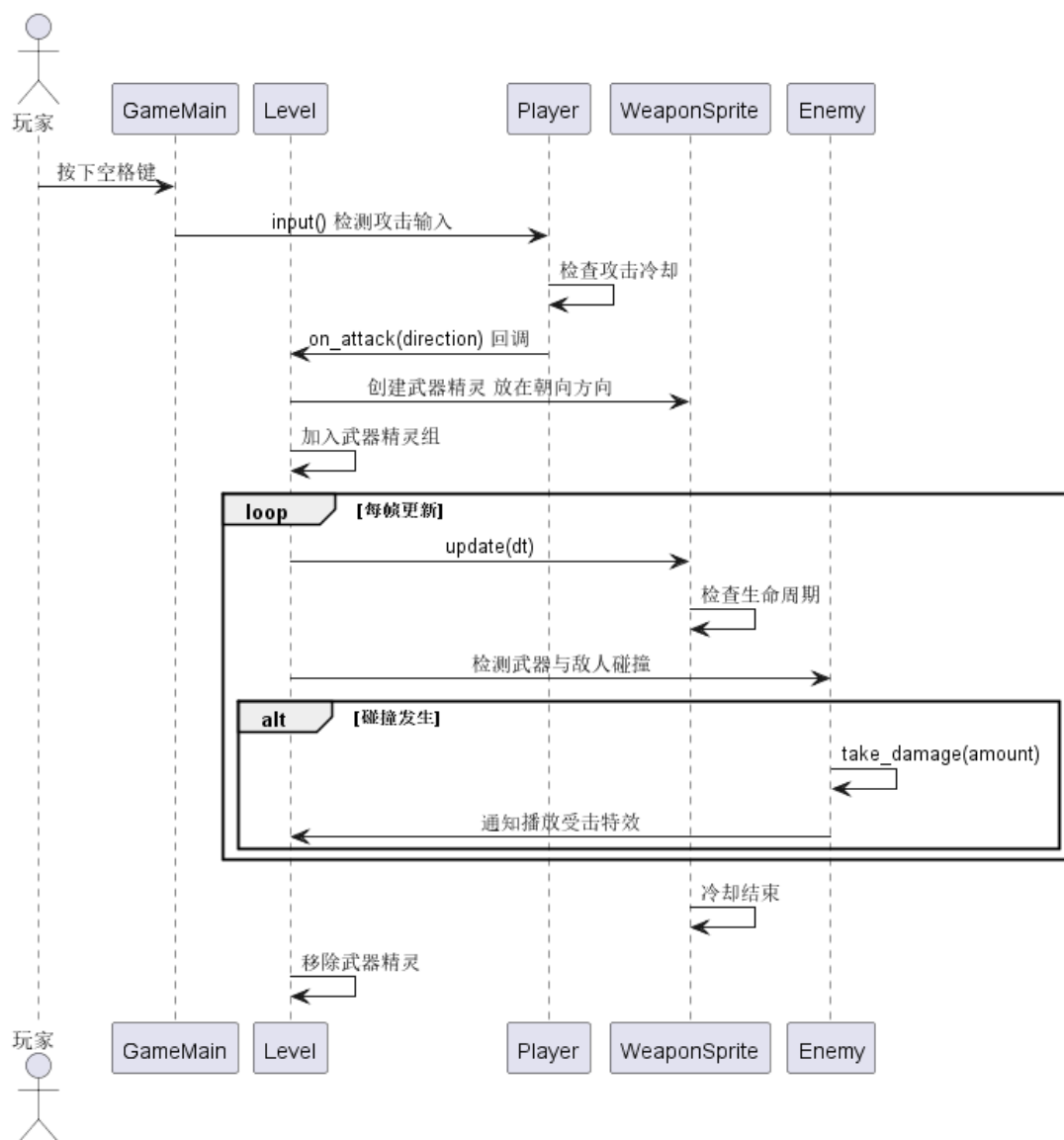


图 5: 近战攻击时序图 (PlantUML 生成)

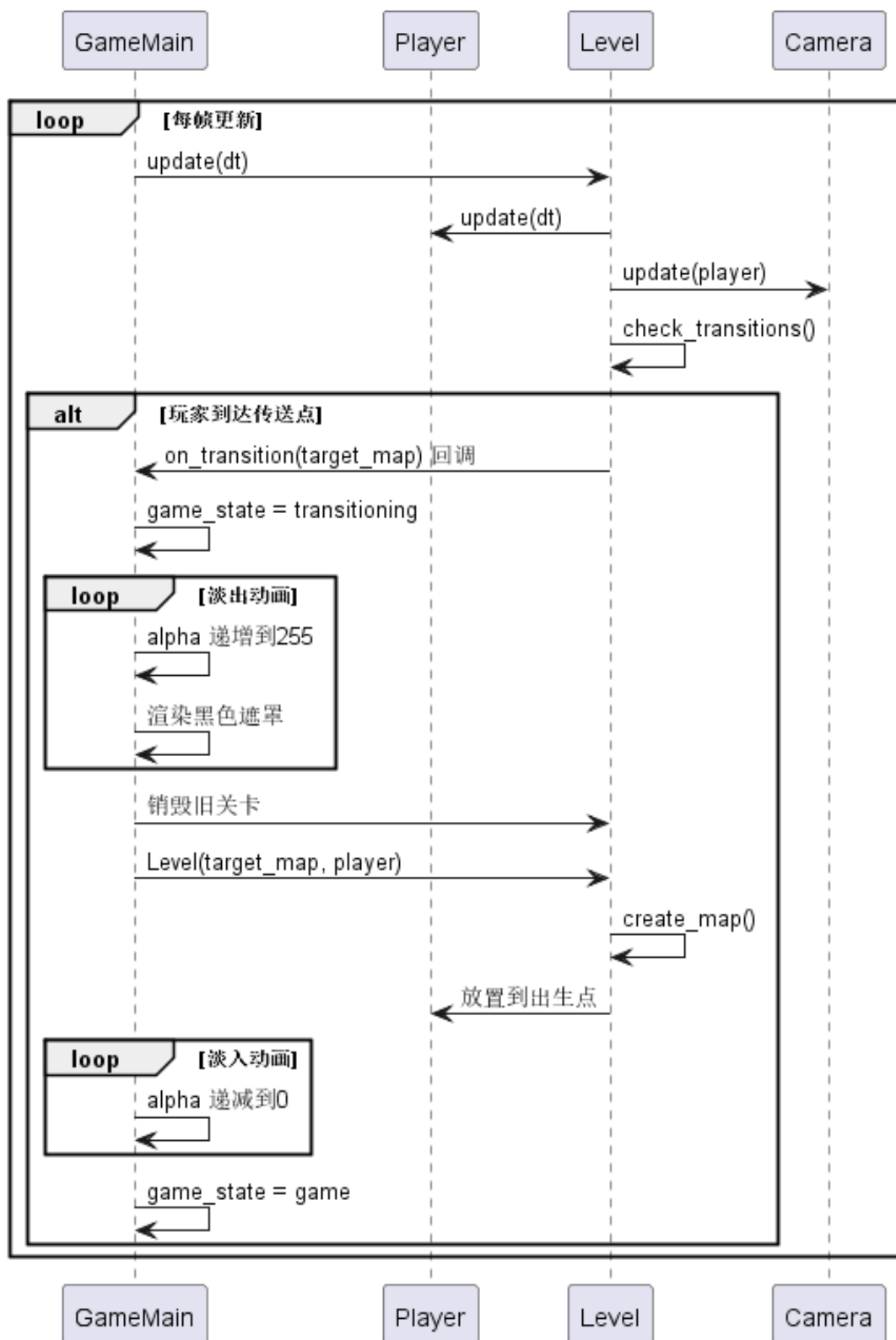
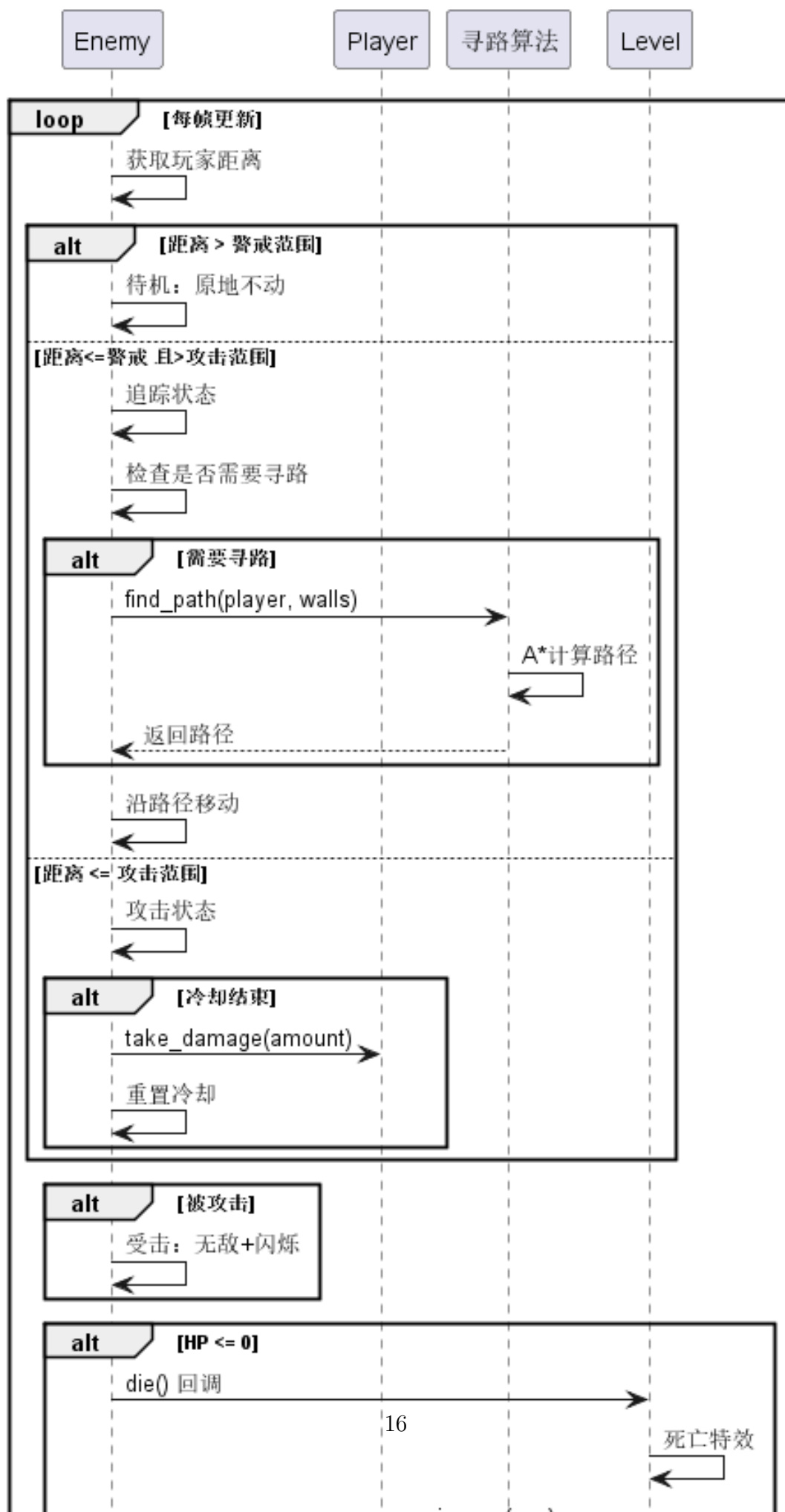


图 6: 场景切换时序图 (PlantUML 生成)



怪物 AI 行为时序图

AI 流程：每帧获取玩家距离 → 判断状态 → 待机/追踪/攻击 → 追踪时调用寻路算法 → 被攻击进入受击状态 → $HP \leq 0$ 则死亡。

第 5 章 数据结构设计

配置数据结构

武器、怪物、魔法数据采用字典嵌套结构，实现数据驱动设计。新增内容无需修改核心代码，只需添加配置条目。

数据类型	结构	字段
武器	dict[str, dict]	damage, cooldown, graphic, sound
怪物	dict[str, dict]	health, damage, speed, exp, attack_type, size, hitbox
魔法	dict[str, dict]	strength, cost, graphic, sound

地图数据结构

每张地图由四个 CSV 文件组成：

文件	数据类型	特殊值
ground.csv	int[][]	各值对应地面瓦片编号
grass.csv	int[][]	1= 可破坏草地
collision.csv	int[][]	0= 可通行, 1= 不可通行, 特殊值 = 出生点/敌人/传送点
entities.csv	int[][]	特殊值标识敌人类型和传送目标

存档数据结构

JSON 格式，序列化玩家状态和世界状态：

字段	类型	说明
player.position	[float, float]	玩家坐标
player.health	int	当前生命值
player.energy	int	当前能量值
player.exp	int	当前经验值
player.stats	dict	五项属性值
player.weapon	str	当前装备武器
current_map	str	当前地图 ID
defeated_enemies	list[[int,int]]	已击败敌人坐标
destroyed_grass	list[[int,int]]	已破坏草地坐标

第 6 章 场景交互设计

场景状态转换图

各场景详细设计

主菜单场景

- 背景渲染：深蓝色渐变（RGB: 20, 20, 40），浮动蓝色粒子
- 标题动画：金色文字，正弦函数浮动 $y = A \sin(\omega t + \phi)$
- 菜单选择：上下键移动高亮，选中项脉冲闪烁（每 100ms 切换）
- 音频：背景音乐循环播放

游戏关卡场景

- 地图渲染：地面 → 草地 → 装饰 → 实体，四层叠加
- 视窗系统：以玩家为中心截取地图局部，计算相机偏移量并限制在地图范围内
- 实体更新：玩家 → 敌人 → 武器 → 魔法 → 粒子，按顺序更新
- 碰撞调度：实体与墙壁碰撞、武器与敌人碰撞、玩家与传送点碰撞
- HUD：屏幕顶部固定绘制血条/能量条/武器图标/经验值

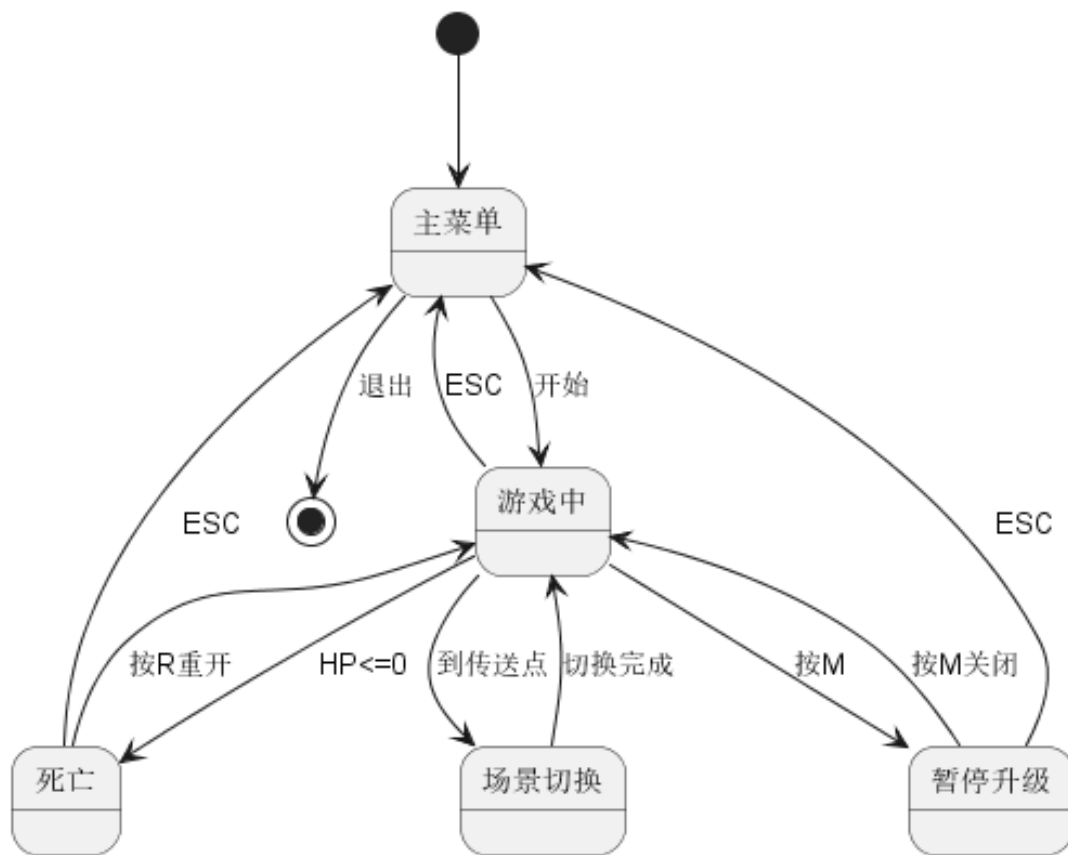


图 8: 场景状态转换图 (PlantUML 生成)

死亡界面

- 背景：暗红色（RGB: 40, 0, 0），红色浮动粒子
- 标题：红色大字「You Died」
- 统计：显示本局获得经验值
- 选项：按 R 重新开始 / 按 ESC 退出

第7章 测试方案

功能测试

编号	测试项	测试内容	预期结果	优先级
FT-01	主菜单显示	启动游戏，检查菜单是否正常显示	3 秒内显示菜单，音乐播放	P0
FT-02	菜单导航	上下键选择，回车确认	选择正确响应	P0
FT-03	玩家移动	四方向移动测试	流畅移动，动画正确	P0
FT-04	碰撞检测	靠近墙壁/障碍物测试	不穿墙	P0
FT-05	近战攻击	空格键攻击，不同武器测试	伤害正确，冷却有效	P0
FT-06	魔法释放	Ctrl 释放治疗和火焰	能量扣除，效果正确	P0
FT-07	武器切换	Q 键切换五种武器	切换正确，属性更新	P0
FT-08	敌人 AI	四种怪物行为测试	待机/追踪/攻击正常	P0
FT-09	智能寻路	怪物绕障碍物追踪	能绕开障碍物	P0

编号	测试项	测试内容	预期结果	优先级
FT-10	地图渲染	多图层叠加显示	渲染正确, 遮挡关系正确	P0
FT-11	场景切换	传送点切换测试	切换流畅, 状态保留	P0
FT-12	存档读档	保存和加载测试	数据完整 正确	P1
FT-13	升级系统	分配经验值测试	属性正确 提升	P1
FT-14	死亡处理	血量归零测试	死亡界面显示, 重开正常	P1
FT-15	帧率测试	复杂场景运行 10 分钟	帧率稳定 60FPS+	P1

边界测试

编号	测试项	测试内容	预期结果	优先级
BT-01	地图边界	移动到地图边缘	不越界	P0
BT-02	存档损坏	使用损坏的存档文件	弹出提示, 允许新游戏	P1
BT-03	资源缺失	删除部分图片资源	游戏不崩溃, 使用占位图	P1
BT-04	快速按键	快速连续按键	不崩溃, 状态正确	P1
BT-05	多敌人同屏	同时存在 10+ 敌人	帧率稳定	P1

编号	测试项	测试内容	预期结果	优先级
BT-06	长时间运行	连续运行 1 小时	无内存泄漏，帧率稳定	P2

测试策略

1. **单元测试**：对核心算法（碰撞检测、寻路、状态机）编写独立测试
2. **集成测试**：模块组合后测试交互（玩家攻击 + 敌人受击 + 特效触发）
3. **系统测试**：完整游戏流程测试（主菜单 → 游戏 → 死亡 → 重开）
4. **性能测试**：监控帧率、内存占用、CPU 使用率

第 8 章 项目统计

类别	数量	说明
Python 源文件	约 16 个/约 2000 行	核心游戏逻辑
地图数据	约 18 个 CSV 文件	4 张地图 × 4 层
图片资源	约 275 个 PNG	角色/怪物/武器/特效/瓦片
音频资源	约 9 个文件	背景音乐 + 音效

参考文献

1. Pygame Official Documentation. <https://www.pygame.org/docs/>
2. Hart, P.E., Nilsson, N.J., Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. 1968.
3. Stardew Valley. ConcernedApe.
4. The Legend of Zelda Series. Nintendo.
5. Millington, I., Funge, J. Artificial Intelligence for Games. 2nd Edition, 2009.
6. 软件项目开发与实践课程资料. 刘畅.